

Mera om division

Division fungera inte alltid i C++ såsom man kunde tro. Slutresultatet är beroende på vilka datatyper de tal som divideras har. Det låter ju lite konstigt, men det är både en fördel och en nackdel. Två olika typer av division kan användas: *heltalsdivision* och *flyttalsdivision*.

Heltalsdivision

Om båda operanderna i en division är heltal (`int`, `long` o.s.v.) kommer slutresultatet även att vara ett heltal. Detta innebär att en eventuell decimaldel kommer att totalt ignoreras. Således är följande uttryck sanna:

```
9 / 4 == 2
(100 / 30) * 30 != 100
```

Det verkar konstigt, men det är så det fungerar. För att råda bot på detta problem bör man använda sig av *flyttalsdivision*.

Flyttalsdivision

Flyttalsdivision fungerar såsom division matematiskt normalt fungerar. Där fås alla decimaler till resultatet. Det enda villkoret för att flyttalsdivision skall användas istället för heltalsdivision är att minst ena av operatorerna måste vara ett flyttal (t.ex. `float`, `double`). Exempelen nedan använder sig av flyttalsdivision:

```
float Resultat = 9.0 / 4.0;
float Resultat = 9 / 4.0;
float Resultat = 9.0 / 4;
float Resultat = (float)9 / 4;
```

I dessa exempel är minst ena av operanderna ett flyttal. I det sista exemplet är 9 inget flyttal, men där används en typomvandling för att omvandla denna till ett flyttal. En konstant är alltid ett flyttal om den har en decimaldel, såsom i 4.0, även om talet matematiskt egentligen är ett heltal.

Precisionsproblem

Flyttal i C++ har inte oändlig precision, så ibland kan man få speciella resultat efter att man dividerar med flyttal (och multiplicerar, adderar och subtraherar) beroende på precisionsproblem. Alltid är t.ex. `10.0 / 5.0` inte exakt `2.0`, utan det *kan* vara t.ex. `2.00000001`. Om dylika fel uppkommer och ackumuleras kan man med tiden få ett fel som är signifikant. Man kan undvika detta problem till en viss del egenom att använda sig av flyttalstypen `double` istället för `float`, eftersom den har dubbel precision jämfört med `float`, och således lider i en mindre grad av ackumulerande fel.

Det går således inte alltid att direkt jämföra ett flyttal med en konstant, eftersom flyttalet kan vara minimalt större eller mindre än konstanten man jämför med. Detta kan vara bra att komma ihåg om man hanterar flyttal och vill jämföra dem med varandra.

Division med 0

Division med 0 är något som man normalt bör undvika. Då man utför divisioner bör nämnarens värde kontrolleras för att undvika division med 0. Beroende på systemet kan detta leda till en omedelbar terminering av programmet, eller så att divisionen resulterar i ett speciellt värde som resulterar ett ogiltigt eller oändligt värde. I vissa fall kan en *exception* kastas (se [Kapitel 20](#)).

[Förutgående](#)
Aritmetik

[Hem](#)
[Upp](#)

[Nästa](#)
Binär aritmetik